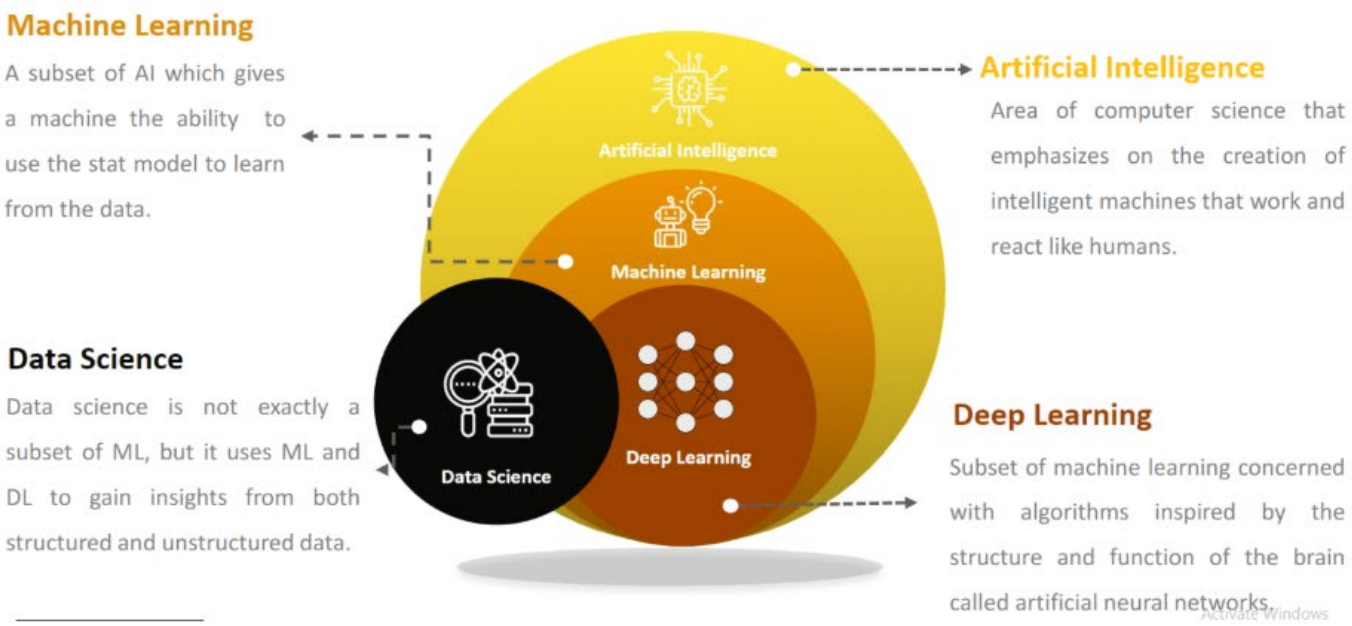
Notes and Exercises Chapter 9 – Logistic Regression
Math 217
Saidi

Chapter 9



Logistic regression is a tool for building models when there is a categorical response variable with two levels, e.g., yes and no. Logistic regression is a type of **generalized linear model (GLM)** for response variables where regular multiple regression does not work very well. GLMs can be thought of as a two-stage modeling approach. **F**irst, model the response variable using a probability distribution, such as the binomial or Poisson distribution. Second, we model the parameter of the distribution using a collection of predictors and a special form of multiple regression. Ultimately, the application of a GLM will feel very similar to multiple regression, even if some of the details are different.

Logistic Regression is used to predict a binary outcome (1 / 0, Yes / No, True / False) given a set of independent variables. To represent binary / categorical outcome, we use dummy variables. You can also think of logistic regression as a special case of linear regression when the outcome variable is categorical, where we are using log of odds as dependent variable. In simple words, it predicts the probability of occurrence of an event by fitting data to a logit function.

The fundamental equation of generalized linear model is:

$$g\big(E(y)\big) = \beta_0 + \beta_1(predictor1) + \beta_2(predictor2)$$

Here, g() is the link function, E(y) is the expectation of target variable and $\beta_0 + \beta_1(predictor1) + \beta_2(predictor2)$ is the linear predictor. The role of link function is to 'link' the expectation of y to linear predictor.

Since probability must always be positive, we'll put the linear equation in exponential form. For any value of slope and dependent variable, exponent of this equation will never be negative.

$$\hat{p} = e^{(\beta_0+\beta(predictor))} \quad \text{------- (b)}$$

To make the probability less than 1, we must divide p by a number greater than p. This can simply be done by:

$$\hat{p} = \frac{e^{(\beta_0+\beta(predictor)}}{e^{(\beta_0+\beta(predictor))}+1} \quad \text{----- (c)}$$

Using (a), (b) and (c), we can redefine the probability as:

$$\hat{p} = \frac{e^{y}}{1+e^{y}} \quad \text{--- (d)}$$

where $\hat{p}$ is the probability of success. *This (d) is the Logit Function.*
If $\hat{p}$ is the probability of success, $1-\hat{p}$ will be the probability of failure which can be written as:

$$\hat{q} = 1-\hat{p} = 1-\frac{e^{y}}{1+e^{y}} \quad \text{--- (e)}$$

where $\hat{q}$ is the probability of failure.

Odds of winning
2:1
$\hat{p}(win) = \frac{2}{3}$
$\hat{p}(lose) = \frac{1}{3}$

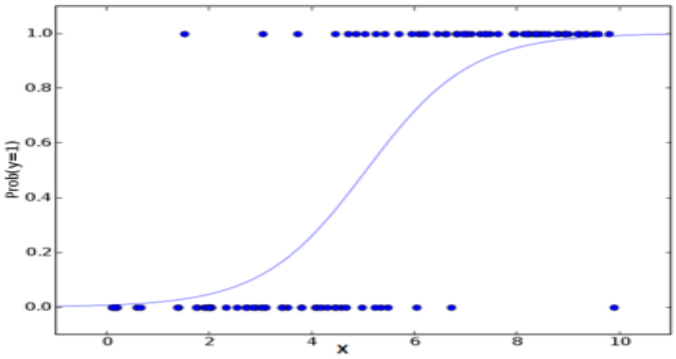
On dividing, (d) / (e), we get, $\left(\frac{\hat{p}}{1-\hat{p}}\right) = e^{y}$

After taking log on both sides, we get $log\left(\frac{\hat{p}}{1-\hat{p}}\right) = y$

$log\left(\frac{\hat{p}}{1-\hat{p}}\right)$ is the **link function.** Logarithmic transformation on the outcome variable allows us to model a non-linear association in a linear way.

After substituting value of y, we'll get: $log\left(\frac{\hat{p}}{1-\hat{p}}\right) = \beta_0 + \beta(predictor)$

This is the equation used in Logistic Regression. Here ($\hat{p}$/1-$\hat{p}$) is the odds ratio. Whenever the log of odd ratio is found to be positive, the probability of success is always more than 50%. A typical logistic model plot is shown below. You can see probability never goes below 0 and above 1.



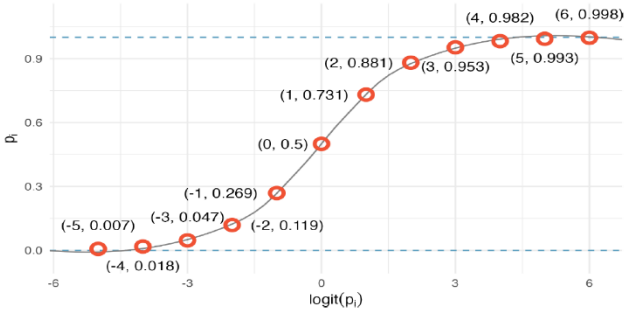
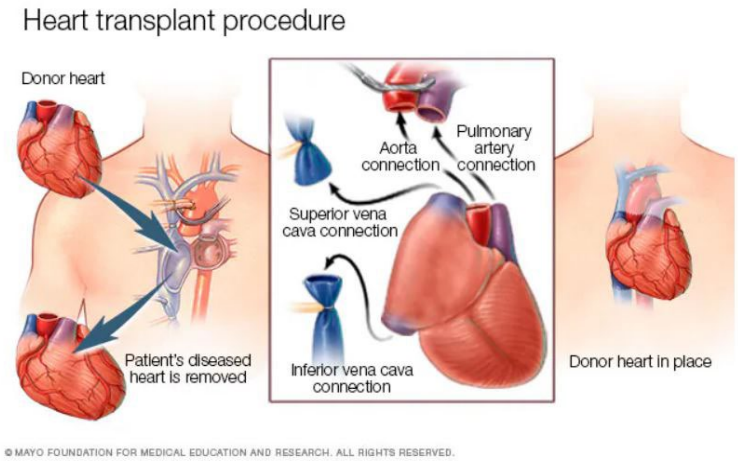


Figure 9.1: Values of p_i against values of $logit(p_i)$.

Chapter 9 logistic regression



https://www.mayoclinic.org/tests-procedures/heart-transplant/about/pac-20384750

Table of contents

Load library tidyverse and heart transplant dataset

Load libraries and use “data” and “head” to view the variables in the dataset

```
library(tidyverse)
library(tidymodels)
library(openintro)
data("heart_transplant")
head(heart_transplant)

# A tibble: 6 × 8
     id acceptyear   age survived survtime prior transplant  wait
  <int>      <int> <int> <fct>       <int> <fct> <fct>      <int>
```

```
1    15          68    53 dead            1 no    control       NA
2    43          70    43 dead            2 no    control       NA
3    61          71    52 dead            2 no    control       NA
4    75          72    52 dead            2 no    control       NA
5     6          68    54 dead            3 no    control       NA
6    42          70    36 dead            3 no    control       NA
```

```
heart <- heart_transplant #rename the dataset with a shorter name
```

Ensure the response variable is binary

```
unique(heart$survived)
```

```
[1] dead  alive
Levels: alive dead
```

List all variables in the dataset

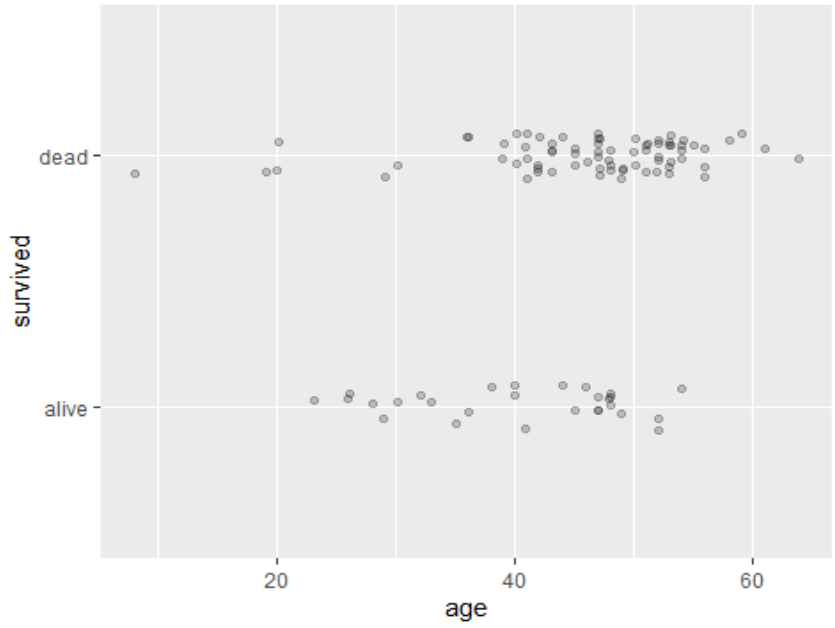
```
names(heart) <- tolower(names(heart)) # makes variables lowercase
names(heart) <- gsub(" ", "", names(heart)) # removes spaces in spaces in variable names
```

Visualizing the data

Let's start by looking at whether an applicant survived the transplant based on years.

Note that in the plot below, we use the **geom_jitter()** function to create the illusion of separation in our data. Because the y value is categorical, all of the points would either lie exactly on "dead" or "alive", making the individual points hard to see. To counteract this, geom_jitter() will move the points a small random amount up or down.

```
ggplot(data = heart, aes(x = age, y = survived)) +
  geom_jitter(width = .09, height = 0.09, alpha = 0.2)
```



Make a binary variable

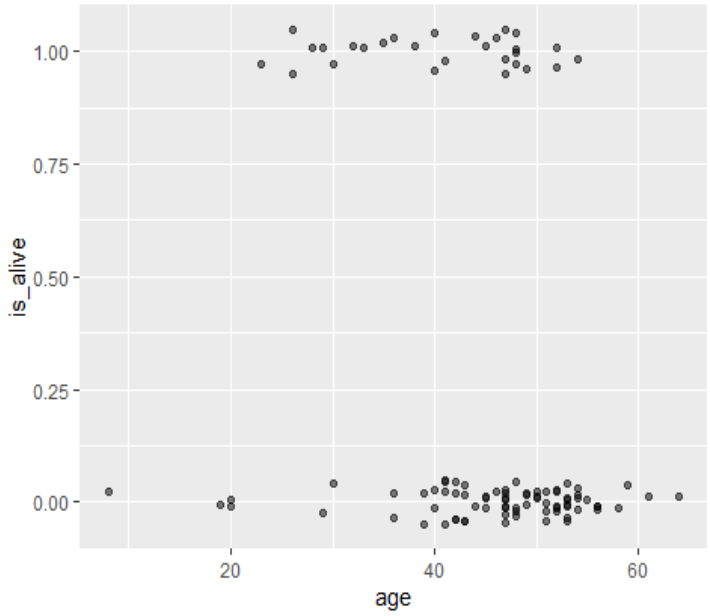
First, we have a technical problem, in that the levels of our response variable are labels, and you can't build a regression model to a variable that consists of words! We can get around this by creating a new variable that is binary (either 0 or 1), based on whether the patient survived to the end of the study. We call this new variable is_alive.

```
heart1 <- heart|>
  mutate(is_alive = ifelse(survived == "alive", 1, 0)) # alive = 0, died = 1
```

Visualize a binary response

We can then visualize our data_space. The vertical axis can now be thought of as the probability of being alive at the end of the study, given one's age at the beginning.

```
ggplot(data = heart1, aes(x = age, y = is_alive)) +
  geom_jitter(width = 0, height = 0.05, alpha = 0.5)
```



Generalized linear models

Thankfully, a modeling framework exists that generalizes regression to include response variables that are non-normally distributed. This family is called generalized linear models or GLMs for short. One member of the family of GLMs is called logistic regression, and this is the one that models a binary response variable.

A full treatment of GLMs is beyond the scope of this tutorial, but the basic idea is that you apply a so-called link function to appropriately transform the scale of the response variable to match the output of a linear model. The link function used by logistic regression is the logit function. This constrains the fitted values of the model to always lie between 0 and 1, as a valid probability must.

In this lesson we will cover: - generalization of multiple regression - model non-normal responses - special case: logistic regression - models binary response - uses logit link function:

$$logit(p) = log\big(p/(1-p)\big) = \beta_0 + \beta_1 * x$$

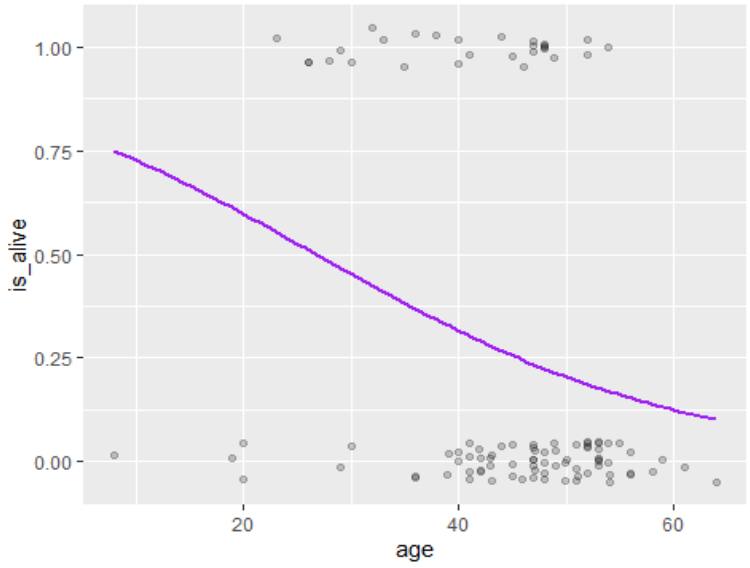
Fitting a GLM

********don't forget the "family = binomial**********

```
fit1 <- glm(is_alive ~ age, family = binomial(), data = heart1)
summary(fit1)


Call:
glm(formula = is_alive ~ age, family = binomial(), data = heart1)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  1.56438    1.01521   1.541   0.1233
age         -0.05847    0.02306  -2.535   0.0112 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 120.53  on 102  degrees of freedom
Residual deviance: 113.67  on 101  degrees of freedom
AIC: 117.67

Number of Fisher Scoring iterations: 4
```

```
plot_log <- ggplot(data = heart1, aes(y = is_alive, x = age)) +
  geom_jitter(width = .05, height = 0.05, alpha = 0.2) +
  geom_smooth(method = "glm", color = "purple", method.args = list(family = "binomial"),
se = FALSE)
plot_log
```

```
`geom_smooth()` using formula = 'y ~ x'
```



geom_smooth(method = "glm") creates a **sigmoid** curve, like a stretched-out S. This curve fits to the binary values (zeros and ones plotted). It shows the predicted probabilities of being alive, based on age.

As an example, a 60 year old patient has an approximately 12-13% probability of survival.

The probability equation is:

$$\hat{y} = \frac{exp(\widehat{\beta_0} + \widehat{\beta_1} * x)}{1 + exp(\widehat{\beta_0} + \widehat{\beta_1} * x)}$$

Here, we compute the fitted probabilities using the augment() function.

Set the type.predict argument to "response" to retrieve the **fitted values** on the familiar probability scale.

```
heart_plus <- fit1 |>
  augment(type.predict = "response") |>
  mutate(y_hat = .fitted)
heart_plus
```

```
# A tibble: 103 × 9
   is_alive   age .fitted .resid   .hat .sigma .cooksd .std.resid y_hat
      <dbl> <int>   <dbl>  <dbl>  <dbl>  <dbl>   <dbl>      <dbl> <dbl>
 1        0    53   0.177 -0.625 0.0156   1.06 0.00174     -0.630 0.177
 2        0    43   0.279 -0.809 0.0106   1.06 0.00210     -0.813 0.279
 3        0    52   0.186 -0.642 0.0147   1.06 0.00173     -0.646 0.186
 4        0    52   0.186 -0.642 0.0147   1.06 0.00173     -0.646 0.186
 5        0    54   0.169 -0.608 0.0166   1.06 0.00175     -0.614 0.169
 6        0    36   0.368 -0.958 0.0181   1.06 0.00548     -0.967 0.368
 7        0    47   0.234 -0.731 0.0111   1.06 0.00174     -0.735 0.234
 8        0    41   0.303 -0.850 0.0115   1.06 0.00257     -0.855 0.303
 9        0    47   0.234 -0.731 0.0111   1.06 0.00174     -0.735 0.234
10        0    51   0.195 -0.659 0.0138   1.06 0.00172     -0.663 0.195
# ℹ 93 more rows
```

Odds scale

To combat the problem of the scale of the y variable, we can change the scale of the variable on the y-axis. Instead of thinking about the probability of receiving a callback, we can think about the odds. While these two concepts are often conflated, they are not the same. They are however, related by the simple formula below. The odds of a binary event are the ratio of how often it happens, to how often it doesn't happen.

odds(y-hat)= y-hat / (1−y-hat) = exp(β_0+β_1·x)

$$odds(\hat{y}) = \frac{\hat{y}}{1 - \hat{y}}$$

Thus, if the probability of surviving is 70%, then the odds of surviving is .7/(1-.7) = .7/.3 = 7:3.

The probability scale is the easiest to understand, but it makes the logistic function difficult to interpret. Conversely the logistic function becomes a line on the log-odds scale. This makes the function easy to interpret, but the log of the odds is hard to grapple with. The odds scale lies somewhere in between.

Odds Ratios

$$OR = \frac{exp\left(\widehat{\beta_0} + \widehat{\beta_1} * (x + 1)\right)}{exp(\widehat{\beta_0} + \widehat{\beta_1} * x)} = exp\beta_1$$

```
exp(coef(fit1))

(Intercept)         age
  4.7797050   0.9432099
```

The equation of the model is:

odds(is_alive) = 4.7797 + 0.9432(age).

Example prediction

predict survival for a 70 year old

```
age_70 <- data.frame(age = 70, transplant = "treatment")

augment(fit1, newdata = age_70, type.predict = "response")

# A tibble: 1 × 3
    age transplant .fitted
  <dbl> <chr>        <dbl>
1    70 treatment   0.0739
```

A 70 year old has a predicted probability of 7.39% survival five years after a heart transplant.

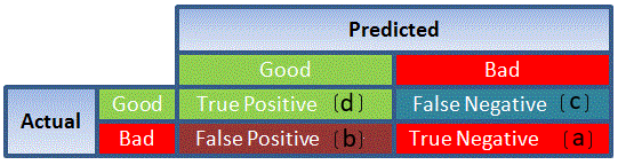
predict survival for a 20 year old

```
age_20 <- data.frame(age = 20, transplant = "treatment")

augment(fit1, newdata = age_20, type.predict = "response")

# A tibble: 1 × 3
    age transplant .fitted
  <dbl> <chr>        <dbl>
1    20 treatment    0.597
```

Confusion matrix

One common way to evaluate the quality of a logistic regression model is to create a confusion matrix, which is a 2×2 table that shows the actual values from the model vs. the predicted values from the testing set.

Confusion Matrix is nothing but a tabular representation of Actual vs Predicted values. This helps us to find the accuracy of the model and avoid overfitting. This is how it looks like:



From the confusion matrix, **Specificity and Sensitivity** can be derived as illustrated below:

As we saw above, accuracy can be defined as:

$$\frac{\text{True Positive + True Negatives}}{\text{True Positive + True Negatives + False Positives + False Negatives}}$$

Create a confusion matrix manually

- is_alive is the observed value
- alive_hat is the predicted value

```
fit_plus <- augment(fit1, type.predict = "response") |>
  mutate(alive_hat = round(.fitted))

fit_plus |>
  select(is_alive, alive_hat) |>
  table()
```

```
        alive_hat
is_alive  0  1
       0 71  4
       1 25  3
```

```
accuracy_new <- (71 +3)/(71+4+25+3)
accuracy_new
```

```
[1] 0.7184466
```

One common way of assessing performance of models for a categorical response is via a confusion matrix. This simply cross-tabulates the reality (is_alive) with what our model predicted (alive_hat).

In this case, our model predicted that 96 patients would be alive, and only 7 would die. Of those 96, 71 actually were alive, while of the 7, 3 actually died. Thus, our overall accuracy was 74 out of 103, or about 72%.

Use the Tidymodels package to create a regression model

Create the training, testing, and validation data by splitting

Initial_split creates a single binary split of the data into a training set and testing set (the default is 75% training 25% testing)

```
set.seed(123)

res_splits<- initial_split(heart1, strata = is_alive)

alive_train <- training(res_splits) #default 75%
alive_test  <- testing(res_splits) # default 25%
```

Calculate training set proportions by is_alive

```
alive_train |>
  group_by(is_alive) |>
  tally() |>
  mutate(prop = n/sum(n))

# A tibble: 2 × 3
  is_alive     n  prop
     <dbl> <int> <dbl>
1        0    56 0.727
2        1    21 0.273
```

We can see that 72.8% are alive versus 27.2% are not alive 5 years after transplant surgery.

This is called "imbalanced", which can lead to biased results. A biased sample can often cause a great amount of disorder to the analysis.

Fitting Logistic Regression

You can fit any type of model (supported by tidymodels) using the following steps.

1. Call the model function: here we called logistic_reg( ) as we want to fit a logistic regression model.
2. Use set_engine( ) function to supply the family of the model. The "glm" argument as Logistic regression comes under the Generalized Linear Regression family.
3. Next, you need to use the fit( ) function to fit the model and inside that in the form y~x, you have to provide the formula notation and dataset (callback_train).

plus notation → diabetes ~ ind_variable 1 + ind_variable 2 + …….so on

```
logit_fit_train <-
  logistic_reg(mode = "classification") |>
  set_engine(engine = "glm") |>
  fit(survived ~ age, data = alive_train)
logit_fit_train

parsnip model object


Call:  stats::glm(formula = survived ~ age, family = stats::binomial,
    data = data)

Coefficients:
(Intercept)          age
    -3.4172       0.1006

Degrees of Freedom: 76 Total (i.e. Null);  75 Residual
Null Deviance:      90.24
Residual Deviance: 77.76    AIC: 81.76
```

This model should be similar to the model we built using glm in the base R.

The AIC (Akaike information criterion): assesses the model in comparison to other models; the lower this value, the better the model.

This first training model shows AIC: 81.76 which is better than the AIC of 117 from the full dataset.

Odds Ratio

The interpretation of coefficients in the log-odds term does not make much sense if you need to report it in your article or publication. That is why the concept of odds ratio was introduced.

The ODDS is the ratio of the probability of an event occurring to the event not occurring. When we take a ratio of two such odds it called Odds Ratio.

you can get directly the odds ratios of the coefficient by supplying the exponentiate = True inside the tidy( ) function.

```
exp(0.1006)
```

```
[1] 1.105834
```

```
tidy(logit_fit_train, exponentiate = TRUE)
```

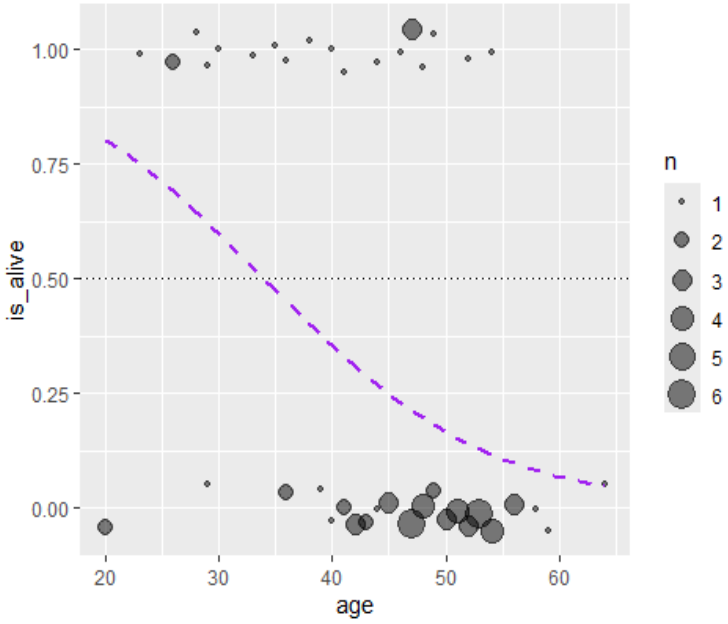
```
# A tibble: 2 × 5
  term        estimate std.error statistic p.value
  <chr>          <dbl>     <dbl>     <dbl>   <dbl>
1 (Intercept)   0.0328    1.37       -2.50 0.0123
2 age           1.11      0.0312      3.23 0.00124
```

Illustrate the predicted probability

Illustrate how the predicted probability of dying varies in our simple logistic regression model with respect to a person's age. Notice the sigmoid curve (in purple) - this is the shape of a logistic curve.

```
ggplot(alive_train, aes(x = age, y = is_alive)) +
  geom_count(
    position = position_jitter(width = 0, height = 0.05),
    alpha = 0.5 ) +
  geom_smooth(method = "glm", method.args = list(family = "binomial"),
              color = "purple", lty = 2, se = FALSE) +
  geom_hline(aes(yintercept = 0.5), linetype = 3)
```

```
`geom_smooth()` using formula = 'y ~ x'
```



This plot is very similar to the plot for the entire data.

Model Prediction

Test Data Class Prediction

The very next step is to generate the test predictions that we could use for model evaluation. To generate the class prediction (pos/ neg) we can use the predict function and supply the trained model object, test dataset and the type which is here "class" as we want the class prediction, not probabilities.

To get predicted values we use the function **predict**.

```
pred_class <- predict(logit_fit_train,
                      new_data = alive_test,
                      type = "class")

pred_prob <- predict(logit_fit_train,
                      new_data = alive_test,
                      type = "prob")

joined_pred_results <- alive_test |>
  select(survived, is_alive) |>
  bind_cols(pred_class, pred_prob)
```

Model performance

yardstick is a package to get metrics on model performance. By default on classification problem yardstick::metrics returns accuracy and kappa.

https://yardstick.tidymodels.org/

```
library(yardstick)
metrics(joined_pred_results, truth = survived, estimate = .pred_class)
```

```
# A tibble: 2 × 3
  .metric  .estimator .estimate
  <chr>    <chr>          <dbl>
1 accuracy binary        0.654
2 kap      binary       -0.0174
```

The accuracy is only 65.3% and the kappa is very low at basically zero. A kappa of 1 means perfect agreement with the two categorical classifications, and a kappa of 0 means no agreement (no better than random assignment).

The Confusion Matrix - Sensitivity and Specificity

The **conf_mat** function automates creating a confusion matrix

```
conf_mat(joined_pred_results, truth = survived, estimate = .pred_class)
```

```
          Truth
Prediction alive dead
     alive     1    3
     dead      6   16
```

```
17/26
```

```
[1] 0.6538462
```

Find the sensitivity and specificity of the model.

```
sens <- sens(joined_pred_results, truth = survived, estimate = .pred_class)
spec <- spec(joined_pred_results, truth = survived, estimate = .pred_class)
rbind(sens, spec)
```

```
# A tibble: 2 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 sens    binary         0.143
2 spec    binary         0.842
```

The sensitivity is too low and the sum of sens and spec is less than 0.98. So, this model does NOT look good as inspected above.

ROC Curve - Receiver Operator Characteristics

Now, we will use the ROC curve along with its corresponding AUC (area under the curve) for the testing set.

ROC - Receiver Operating Characteristics

Receiver Operating Characteristics Curve traces the percentage of true positives accurately predicted by a given logit model as the prediction probability cutoff is lowered from 1 to 0. For a good model, as the cutoff is lowered, it should mark more of actual 1's as positives and lesser of actual 0's as 1's. So for a good model, the curve should rise steeply, indicating that the TPR (Y-Axis) increases faster than the FPR (X-Axis) as the cutoff score decreases.

AUC - Area under the curve

The greater the area under the ROC curve, better the predictive ability of the model

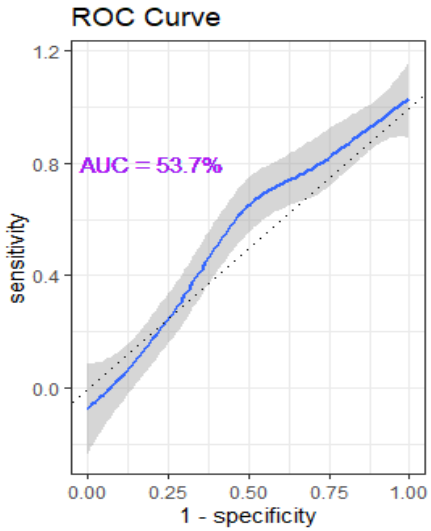
```
roc_auc(bind_cols(alive_test, pred_prob), survived, .pred_dead)
```

```
# A tibble: 1 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 roc_auc binary         0.538
```

The AUC estimate is 55% - meaning the model does a moderate job of predicting survival 5 years after a heart transplant, based on age.

```
roc_data <- roc_curve(bind_cols(alive_test, pred_prob), truth = survived, .pred_dead)

roc_data |>
  ggplot(aes(x = 1 - specificity, y = sensitivity)) +
  geom_smooth() +
  geom_abline(lty = 3) +
  coord_equal()+
  ggtitle("ROC Curve")+
  theme_bw()+
  geom_text(x=0.2, y=.8, label="AUC = 53.7%", color ="purple")
```

```
`geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



The Full Regression Model

Computing the full logistic regression model will feel similar to the process for computing the full multiple linear regression model. We will use **GLM** to replace **LM** and we need to include **family = binomial()**

```
log_mod_full <- glm(data=alive_train, is_alive ~ acceptyear + age + survtime + prior + tr
ansplant, family = binomial())
summary(log_mod_full)
```

```

Call:
glm(formula = is_alive ~ acceptyear + age + survtime + prior +
    transplant, family = binomial(), data = alive_train)

Coefficients:
```

```
                      Estimate Std. Error z value Pr(>|z|)
(Intercept)         -217.83596   77.09940  -2.825  0.00472 **
acceptyear             3.08538    1.07644   2.866  0.00415 **
age                   -0.17879    0.07729  -2.313  0.02070 *
survtime               0.01046    0.00336   3.113  0.00185 **
prioryes              -1.22206    1.22549  -0.997  0.31867
transplanttreatment   -1.27372    1.47948  -0.861  0.38928
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 90.237  on 76  degrees of freedom
Residual deviance: 26.020  on 71  degrees of freedom
AIC: 38.02

Number of Fisher Scoring iterations: 9
```

Consider Survival of Transplant Under Treatment versus Control

Notice in the above model, the p-value is large at .38928

Ho: Survival is independent of the treatment or control Ha: “”””” is not independent “””

```
chisq.test(alive_train$transplant, alive_train$survived)


    Pearson's Chi-squared test with Yates' continuity correction

data:  alive_train$transplant and alive_train$survived
X-squared = 0.98226, df = 1, p-value = 0.3216
```

*The p-value is also large for the Chi Square Test, so we can remove “transplant”

Interestingly, if you create a model using all the data, the results are different. See below.

```
log_mod_full <- glm(data=heart1, is_alive ~ acceptyear + age + survtime + prior + transpl
ant, family = binomial())
summary(log_mod_full)


Call:
glm(formula = is_alive ~ acceptyear + age + survtime + prior +
    transplant, family = binomial(), data = heart1)

Coefficients:
                      Estimate Std. Error z value Pr(>|z|)
(Intercept)         -1.709e+02  4.774e+01  -3.581 0.000343 ***
acceptyear           2.378e+00  6.560e-01   3.624 0.000290 ***
age                 -9.084e-02  4.831e-02  -1.880 0.060075 .
survtime             9.018e-03  2.267e-03   3.978 6.95e-05 ***
prioryes            -1.233e+00  9.628e-01  -1.280 0.200440
transplanttreatment -3.712e-01  1.109e+00  -0.335 0.737831
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 120.528  on 102  degrees of freedom
Residual deviance:  42.804  on  97  degrees of freedom
AIC: 54.804

Number of Fisher Scoring iterations: 8
```

```
chisq.test(heart1$transplant, heart1$survived)
```

```
    Pearson's Chi-squared test with Yates' continuity correction

data:  heart1$transplant and heart1$survived
X-squared = 4.9891, df = 1, p-value = 0.02551
```

This likely means that there is a relationship between survival and the treatment, but not when in conjunction with other variables.

Performance of Logistic Regression Model

To evaluate the performance of a logistic regression model, we must consider few metrics: AIC and p-values.

Assessing our full model

4. The AIC is 35.871 (recall it was 67.79 in the simple model with only age as a predictor)
5. The largest p-values come from transplant and prior

Remove transplant and prior and re-run the model

```
log_mod2 <- glm(data=alive_train, is_alive ~ acceptyear + age + survtime, family = binomi
al())
summary(log_mod2)
```

```
Call:
glm(formula = is_alive ~ acceptyear + age + survtime, family = binomial(),
    data = alive_train)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.979e+02  7.040e+01  -2.811  0.00494 **
acceptyear   2.809e+00  9.871e-01   2.846  0.00443 **
age         -1.961e-01  7.927e-02  -2.474  0.01338 *
survtime     9.028e-03  2.903e-03   3.110  0.00187 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 90.237  on 76  degrees of freedom
Residual deviance: 28.278  on 73  degrees of freedom
AIC: 36.278

Number of Fisher Scoring iterations: 8
```

The AIC decreased to 36.278 and all p-values are relatively low.

VIF - Variance Inflation Factor

Like in case of linear regression, we should check for multicollinearity in the model.

One way to measure multicollinearity is the variance inflation factor (VIF), which assesses how much the variance of an estimated regression coefficient increases if your predictors are correlated.

A VIF between 5 and 10 indicates high correlation that may be problematic. And if the VIF goes above 10, you can assume that the regression coefficients are poorly estimated due to multicollinearity.

```
#install.packages("car")  #car package will calculate the VIF
library(car)
vif(log_mod2)
```

```
acceptyear        age   survtime
  6.574567   1.546562   6.325783
```

Since acceptyear and survtime have VIF values >5, remove one of the two.

```
log_mod3 <- glm(data=alive_train, is_alive ~ survtime + age, family = binomial())
summary(log_mod3)
```

```

Call:
glm(formula = is_alive ~ survtime + age, family = binomial(),
    data = alive_train)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  2.7246374  1.4205899   1.918  0.05512 .
survtime     0.0023885  0.0007661   3.118  0.00182 **
age         -0.1050574  0.0334520  -3.141  0.00169 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 90.237  on 76  degrees of freedom
Residual deviance: 65.556  on 74  degrees of freedom
AIC: 71.556

Number of Fisher Scoring iterations: 5
```

Although all variables have small p-values, the AIC went up.

This tutorial does not come up with any definitive answer about a best model. Instead, it explores the many options and elements to include in a model and how to assess the model.

This tutorial does not come up with any definitive answer about a best model. Instead, it explores the many options and elements to include in a model and how to assess the model.

Homework Chapter 9

1. Review section 9.5 (the chapter review)
2. Required problems from textbook section 9.6 exercises: 3,4,5,6,8
3. Optional Unit 3 Tutorial Regression Modeling:

   9 - Logistic regression